FusePlan – Presentation script

Separating Complementarity from Ensemble Effects in Partial Floor Plan Retrieval

How to use it. One section per slide, in the order of the deck. Each section has: what is on the slide; *what to say*, written in full so you understand what we did, where and why; and the questions that slide can raise. The only formulas are the paper's own (fusion, gain and CG, AUC, plus the pooling and whitening of the vision branch). Numbers follow the slides (AUC ×100, "points").

The thread, in the order of the deck. Partial-plan retrieval is a new task (slide 2) → we solve it with an image branch, a graph branch and a late fusion, plus an ensemble control (3) → the graph is strong and every fusion improves on it (4) → how much fusion helps depends on the relative strength of the branches (5) → the control shows what the image really adds (6) → what we conclude (7).

Suggested split (two slides each, the first speaker also opens with the title): slides 2–3, 4–5, 6–7. Slide 3 is the densest.

Slide 1 – Title

On the slide. FusePlan; the subtitle; the three authors, University of Modena and Reggio Emilia.

What to say. FusePlan retrieves floor plans from *incomplete* queries by fusing two views of a plan, its image and its graph of rooms. The subtitle is our question: when the fusion improves the result, how much of the gain is *complementarity* (the image brings information the graph lacks) and how much is an *ensemble effect* (any two models combined do better than one)?

Slide 2 – Problem and related work

On the slide. Left: problem, goal, question. Right: a table of related work, one line each, and the gap.

What to say

Problem. An architect rarely starts from a complete plan: they sketch a few rooms or reuse a fragment, then look for complete plans that contain it. Existing floor-plan retrieval methods train and evaluate on complete plans; we study retrieval from incomplete ones.

Goal. From a partial plan, find its source among the 67,405 plans of RPLAN. Concretely: we take a plan, remove part of its rooms, and the system must rank the whole collection so that the intact original comes as high as possible. We call this *self-recovery*: the right answer is known by construction, so no human relevance labels are needed.

Why two views, and why it needs care. A plan exists as an image and as a graph of rooms, and they fail differently when rooms are missing. In the image, a removed room leaves a hole inside the outline of the plan: the image still says "something of this shape was here". In the graph, the removed rooms simply disappear, and what is left looks like the complete graph of some smaller plan. So the image *might* add information. But combining any two models whose errors are partly independent improves the result, even if the second one knows nothing new: two trainings of the same network already make different mistakes.

Question. Does the image add information to the graph, or does it only act as a second model?

Related work (one sentence each, as on the slide):

- *LayoutGMN*: graph matching between two complete plans. *LayoutGKN*: graph kernel on complete plans; it is the closest graph method, and we retrain it on our task as baseline. *SSIG*, *Unit Region Encoding*: similarity of complete plans. All evaluate on complete plans.
- *Graph2Plan*, *PlanCraft*: partial plans appear, but for *generating* plans, not retrieving them.
- *On-the-fly sketch retrieval*: retrieves a photo while the sketch is being drawn, so the query is

incomplete; our AUC metric follows theirs.
- *CrossOver*, *MMFE*: multimodal representations learned end to end; they report only the fused accuracy, so they cannot tell new information from an ensemble effect.

Gap: no retrieval method takes partial plans as queries (in our literature search, not exhaustive).

If asked

Q: Why self-recovery and not "find similar plans"?

Similarity needs relevance labels that RPLAN does not have, and any score derived from the metadata is circular for our branches (slide 4). With self-recovery the correct answer is known, so the metric is objective.

Q: Why not compare with CrossOver or MMFE directly?

They solve a different problem (joint embeddings of several modalities of complete scenes or plans) and do not take partial plans as queries. Our point about them is methodological: they report only the fused accuracy.

Slide 3 – Method

On the slide. Pipeline: plan image → frozen PE-Spatial with GeM → PCA whitening → 768-d vector; room graph → GATv2 trained on damaged–complete plans → 128-d vector; late fusion with $\alpha = 0.4$, 896-d, one exact FAISS search over 67,405 plans. Below: graph branch, vision branch, complementarity gain.

What to say

Overall. Two independent branches turn every plan into a unit-length vector. Both index the same gallery of complete plans and receive the *same* removed rooms, so every system can be compared query by query. Similarity is the cosine; for unit vectors it equals the inner product, so we search with an exact inner-product FAISS index, which compares the query with every gallery plan. With 67k plans this is fast, and exact search leaves no approximation error in the comparisons.

Vision branch (top row). The plan's colour rendering is resized (longer side 224 pixels, aspect ratio kept) and padded to a square with the white background. It goes through **PE-Spatial**, the spatially aligned variant of Meta's Perception Encoder (a ViT-B/16), used *frozen*: no training. The encoder gives one vector per image patch; we pool them into one vector with **GeM**, a generalised mean computed per channel, $\left(\frac{1}{N}\sum_i \max(x_i, \epsilon)^p\right)^{1/p}$ with $p = 3$, which gives more weight to the strongest activations than a plain average. One detail: the $\max(\cdot, \epsilon)$ comes from GeM's original use on CNN features, which are never negative; ViT features are signed, so the clamp zeroes the negative values and the descriptor pools only the positive part of each channel. We kept it because it is the configuration that won on validation.

Then **PCA whitening**: $x \mapsto \Lambda^{-1/2}U^\top(x - \mu)$, with mean and covariance estimated on training plans only, keeping all 768 components, then L2 normalisation. In words: centre, rotate onto the principal axes, give every axis unit variance. It matters because every rendering has the same white background, black walls and few colours: without whitening all plans look alike to the encoder and a damaged query matches almost anything. Centring removes what all plans share; rescaling gives the small directions, where plans differ, the same weight as the large common ones. The result is a 768-d vector (768 is the width of a ViT-B).

Optional damage-trained heads. To test whether an image branch that *learns the damage* carries more information, we trained two small heads on the frozen vectors. They are *alternatives*: each replaces the frozen vector in its own fusion; they are never stacked, even if the slide labels read "+ PE-Spatial + SDH".

- **Single-damage head (SDH):** an MLP 768 → 1024 → 256, applied to queries and gallery, trained

to map a plan with rooms removed onto its complete version; its output is 256-d, re-whitened.

- **Multi-damage head (MDH):** acts on the query only, adding a small learned correction to the frozen vector; the gallery stays frozen. Its last layer starts at zero, so at the start it *is* the frozen branch and moves away only where training helps. It trains on three damages (room removal, crop, patch).

Both are trained contrastively (pull a damaged view and its complete plan together, push the other plans of the batch apart), without relevance labels.

Why PE-Spatial. We compared 8 frozen encoders, 52 configurations (slide 5), chosen on validation. PE-Spatial with GeM and whitening was the most robust. *(Interpretation, not measured: in a flat synthetic drawing what matters is* where *things are, which an encoder trained for spatial features captures better.)*

Graph branch (bottom row). The graph is built from the RPLAN *record*, not from the image. Each room is a node, described by its type (13 types) and the geometry of its box (centre, width, height, area, aspect ratio). Each adjacency between rooms is an edge, with its type (10 types, e.g. door or wall). The encoder has three **GATv2** layers: at each layer every room collects information from its neighbours, weighting each neighbour with a learned attention score that depends on both rooms and on the type of their adjacency. Then we *sum* the room vectors into one plan vector, add the plain count of room types (a skip connection), project to 128 dimensions and normalise.

Why. sum, not mean: the sum keeps the number of rooms, a strong clue. The skip connection hands the room counts directly to the output; removing it hurt every encoder. GATv2 over GCN and GraphSAGE: same recipe for the three, GATv2 best, GraphSAGE close, GCN far behind (slide 5); GATv2 is also the only one that reads the edge type.

How the graph is trained. With a contrastive loss (InfoNCE) on *asymmetric* pairs: one view is the complete plan, the other is the same plan with a random 25–75% of its rooms removed, redrawn at every step. For each damaged plan, the network must pick its complete version among the other plans of the batch. This is exactly the task: map an incomplete plan onto the complete one. Recipe: temperature 0.1, batch 256, 600 epochs, training plans only.

Why. with the usual symmetric pairs (two lightly perturbed copies of a plan), every GNN recognised complete plans but failed on damaged ones almost completely: it learned invariance to small perturbations, not to missing rooms.

How the checkpoint is selected ("selected on robustness"). After every epoch we run a small retrieval test: 2,000 validation queries, separate from the evaluation ones, searched against the whole gallery; we keep the epoch with the best robustness. This also tells us there is no overfitting. Overfitting would show as training loss still falling while validation retrieval falls. Instead, in the training log, training and validation loss fall together with a small constant gap, and validation robustness rises and then flattens without a downward trend; in all four replicas the best epoch is at the very end (550–592 of 600). If anything the graph is slightly under-trained, so its score is a lower bound. Random removals redrawn at every step (it never sees the same pair twice), a small model and weight decay protect it. The heads are selected the same way, on 1,000 validation queries; the multi-damage head *did* start to overfit after about 20 epochs (validation robustness down while training loss kept falling) and early stopping kept the best epoch.

Why. select on retrieval, not on the loss: for the graph the validation loss even ranked configurations in the *opposite* order of retrieval. The loss treats every other plan of the batch as a negative, including plans with the same rooms as the anchor, which retrieval would consider close. Selecting on complete plans also failed: it picked early, erratic epochs.

Late fusion (right).

Fusion (paper, Eq. 1)

$$z = [\sqrt{\alpha}\, v\,;\, \sqrt{1-\alpha}\, g], \qquad \langle z_q, z_p \rangle = \alpha \langle v_q, v_p \rangle + (1-\alpha) \langle g_q, g_p \rangle$$

We concatenate the two unit vectors, scaled by $\sqrt{\alpha}$ and $\sqrt{1-\alpha}$. The inner product of two such vectors is the sum of the inner products of their parts, and the square roots on the two sides multiply to α and $1-\alpha$. So the similarity between query q and gallery plan p is a weighted sum of the image cosine and the graph cosine, and the fused vector still has length 1. That is why *one* FAISS search over the 896-d concatenation ranks the gallery by the fused similarity: no need to search the branches separately and merge rankings. At $\alpha = 1$ or 0 it gives back the single branches, a correctness check.

How α is chosen. On validation, over $0, 0.1, \ldots, 1$. Neighbouring weights often score almost the same, so a plain arg-max would pick by chance. Rule fixed in advance: every weight not significantly worse than the best counts as tied, and we take the median of the tied ones. With the frozen branch this gives 0.4 in all four replicas. Every other fusion picks its own weight with the same rule; the test always uses the weights fixed on validation. "Late" fusion means each branch is trained alone and only their similarities are combined: the branches stay independent, so one can be swapped for a control.

Complementarity gain (bottom right). The key idea. Besides fusing the graph with the image, we fuse it with a second model that holds the *same* information as the graph: a second training of the same network, or a GraphSAGE encoder trained the same way. Whatever these *ensemble controls* gain cannot be new information.

Gain and complementarity gain (paper, Eq. 2)

$$\mathrm{gain}_F(q) = \mathrm{AUC}_F(q) - \mathrm{AUC}_b(q), \qquad \mathrm{CG} = \overline{\mathrm{gain}}_{V+G} - \overline{\mathrm{gain}}_{C^\star}$$

The gain of a fusion is its score minus that of its better branch b, per query. The CG is the image–graph gain minus the gain of the *stronger* control $C^\star$: the part of the gain that a second graph model does not reach. $\mathrm{CG} > 0$: the image brings something of its own. $\mathrm{CG} \leq 0$: the gain is explained by having two models. Taking the stronger control is the conservative choice: it makes it harder to claim that the image helps.

If asked

Q: Do all vision encoders give 768-d vectors? Is the graph always 128-d?

No: 768 is PE-Spatial (a ViT-B); larger encoders give wider vectors; the single-damage head gives 256. Every graph encoder we trained outputs 128.

Q: Why a frozen encoder?

The question is what an image representation adds; a frozen encoder answers it without the confound of training. The heads then test, cheaply, what training on the damage adds.

Q: How do you know the training of the GNN and of the heads did not overfit?

After every epoch we measured retrieval on held-out validation queries (2,000 for the graph, 1,000 for the heads, separate from the evaluation queries, against the whole gallery) and kept the best epoch: overfitting would show as training loss still falling while validation retrieval falls. *GNN:* training and validation loss fall together with a small constant gap, validation robustness rises and then flattens with no downward trend, and the best epoch is at the very end (550–592 of 600) in all four replicas; if anything it is slightly under-trained. Removed rooms redrawn at every step, a small model and weight decay protect it, and test values match validation. *Heads:* the multi-damage head did start to overfit after about 20 epochs (validation robustness down, training loss still down) and early stopping kept the best epoch; the single-damage head stopped improving on validation while its validation loss was still falling, so selecting on the loss would have trained it too long. That is why we select on retrieval, not on the loss.

Q: What does the temperature 0.1 do?

It makes the contrastive loss concentrate on the hardest negatives, the plans most similar to the anchor: the fine distinctions retrieval needs. It was the strongest single change for GATv2 and GraphSAGE.

Q: Why is there no per-query weight (gate)?

We use one global α. A gate that picks one branch per query cannot beat the per-query oracle, and our

static fusion already beats it, because summing similarities can push the source above the competitors of *both* branches. A learned gate is future work.

Q: Why two graph models as control?

They hold exactly the graph's information and nothing more, so their gain is the ensemble effect alone.

Slide 4 – Experiments

On the slide. Left: query, metric, protocol, baselines, comparison. Right: test AUC ×100 of single branches and fusions; GATv2-comb beats LayoutGKN by 29 points; every fusion improves on GATv2-comb, mostly at 75% removal.

What to say

Data. RPLAN residential plans: each has a colour rendering (rooms coloured by category) and a record (13 room types, room boxes, typed adjacencies); 4 to 8 rooms per plan. The gallery holds the 67,405 plans with both image and record, in the official training/validation/test split. The gallery always contains every plan, as a search engine would; only what is *learned* (weights, whitening statistics) uses training plans.

Query. A validation or test plan with 25, 50 or 75% of its rooms removed. We remove round($f \cdot n$) of the n rooms: with 7 rooms, 2, 4 and 5. Which rooms is drawn with a seed fixed per query and shared by the two branches, so image and graph lose exactly the same rooms. In the image the removed rooms are painted white and their walls erased; in the graph we keep the remaining rooms and their edges. The vision branch also gets two pixel damages, *crop* (blank a rectangle) and *patch* (blank image patches), used only to choose vision configurations; branches and fusions are compared on room removal, the only damage both can receive.

Metric.

Robustness AUC (paper, Eq. 3)

$$\mathrm{RR}(q, f) = \tfrac{1}{r} \ (r \le 100, \ \text{else } 0)$$

$$\mathrm{AUC}(q) = \frac{1}{3} \sum_{f \in \{.25,.5,.75\}} \mathrm{RR}(q, f), \qquad \mathrm{AUC} = \frac{1}{|Q|} \sum_{q} \mathrm{AUC}(q)$$

For each query and level, the reciprocal rank of the source: 1 if first, 1/2 if second, and so on. We average over the three levels, which is the area under the curve "self-recovery vs fraction removed", then over the queries. 1 means the source is always first; the slide shows it ×100. It follows the metric of on-the-fly sketch retrieval, with "fraction removed" in place of "fraction of the sketch drawn". There is no 0% level: about 3% of RPLAN images have an identical twin, which caps the undamaged score near 0.97 for every system.

Protocol. Every choice (configurations, checkpoints, fusion weights) is made on 2,000 validation queries. Before reading the test split (2,000 queries) we wrote down hypotheses, predicted outcomes and decision rules (*pre-registration*), then read it once with everything fixed. Reason: any choice made after looking at the test set inflates exactly the numbers being reported. Honest caveat: earlier stages of the project, with a weaker graph, also read the test split; nothing reported is selected from those readings.

Statistics. Since all systems see the same queries, we compare two systems query by query: the per-query difference cancels the difficulty both share. For its 95% interval we resample the 2,000 queries with replacement 10,000 times (paired bootstrap); an interval that excludes zero means the difference does not depend on which queries were drawn. Training randomness is covered by repeating the whole pipeline in four replicas with different seeds: an effect counts if it has the same sign in all four and is larger than twice its spread.

Baselines. A room-type histogram without training, and LayoutGKN retrained on our pairs with its

own hyperparameters (with random weights it scores essentially zero, so the training matters).

Results (read the bars).

- The histogram finds almost nothing: counting rooms is not enough.
- Vision alone: frozen PE-Spatial 39.6, with the single-damage head 53.3.
- LayoutGKN 61.1; our graph **GATv2-comb** (called W in the paper) 90.3: 29 points more. The gap is at heavy damage: with a quarter of the rooms missing the two are close; with three quarters missing LayoutGKN collapses while ours still ranks the source near the top. Training and selecting for incompleteness makes the difference.
- Every fusion improves on the graph: 91.6 with the frozen image, 93.4 with the single-damage head, 94.1 with the multi-damage head. Almost all of the gain comes at 75% removal, the only level where the graph leaves room to improve.

Transition. A gain of about one point with the frozen image does not tell, by itself, whether the image helps or whether any second model would. Slides 5 and 6 answer it.

If asked

Q: Is LayoutGKN compared fairly?

Retrained on the same damaged–complete pairs, whole training split, its own hyperparameters, scored with our protocol. Trained once, so its value is a lower bound.

Q: What is a per-axis relevance score, and why is it not used?

On complete plans we describe rankings with nDCG@10 on composition, topology and geometry computed from the metadata. They are circular: the graph receives the same room types and adjacencies they measure (and the image shows the 13 types as 6 colours). So they only describe, never select.

Slide 5 – Relative strength and ablation

On the slide. Left: the inverted-U plot (fusion gain against graph strength; orange frozen vision, green vision with head, filled points on test). Right: graph ablation (52 → 75 → 91) and vision ablation (raw → whitened). Bottom: the families compared, 37 graph configurations × 4 seeds, 52 vision configurations × 3 damages.

What to say

The plot. Each point is one of the 37 graphs of our configuration search, from the weakest to the final one, fused with a vision branch. On the x axis the AUC of the graph alone, on the y axis the fusion gain. The curve is an inverted U:

- graph much weaker than the image: the image does the work, the graph adds little;
- branches equally strong: the gain peaks, around 20 points, and here even the frozen image passes the ensemble controls clearly;
- graph near the ceiling: little is left to fix, and the gain shrinks to about one point with the final graph.

The star is an earlier version of our pipeline, with a weaker graph, which measured a gain of about 17 points: it lies on the same curve, so the old large gain and the new small one do not contradict each other. The message: the fusion gain is not a property of the image, it depends on how strong the graph is relative to it. One detail of the green points: with the two weakest graphs the head *lowers* the gain, because there the head becomes the better branch and raises the reference the gain is measured from.

Graph ablation. GCN, GraphSAGE and GATv2 with the same recipe. From a reference configuration we changed one factor at a time (temperature, skip connection, rotation/reflection augmentation, depth, width, pooling…), kept the changes that helped in all four seeds, and combined them; everything on

validation. For GATv2, temperature 0.1 alone takes 52 to 75; the combination reaches 91. Combining matters more than any single change, above all at heavy damage. GraphSAGE ends close to GATv2, GCN far behind.

Vision ablation. Eight frozen encoders (DINOv2, DINOv3, I-JEPA, SigLIP2, TIPSv2, C-RADIO, PE-Core, PE-Spatial), 52 configurations, selected on the mean of three damages. Whitening helps every encoder in every setting (PE-Spatial 29 → 52). Newer or larger encoders do not win: TIPSv2, the best on complete plans in our benchmark, ranks sixth under damage. The best configuration changes from damage to damage, which is why we select on their average.

Negative result worth telling (if time allows). Our first room-removal rendering erased the dark outer walls but kept the light inner ones, so each removed room stayed as an empty outlined cell: a free hint. It almost doubled the score of a frozen encoder, and a head learned to exploit it. A pre-registered check on other damages exposed it; this is why vision configurations are selected on three damages.

If asked

Q: Why does the gain peak at equal strength?

Two predictors help each other most when neither dominates; when one is near perfect, the other mostly adds noise and can fix only its few remaining errors.

Q: Why is GCN so weak?

It averages a room with its neighbours with fixed weights. *(Interpretation, not measured: this blurs the room's own type, the signal that identifies the plan; GraphSAGE keeps the node separate and GATv2 weighs each neighbour.)*

Q: Does the single-damage head work on other damages?

No: it improves room removal and loses on crop and patch. A head learns the damage it sees, which is why we built the multi-damage head.

Q: Other refuted predictions?

We predicted GraphSAGE to win the search, a much larger frozen fusion gain, and the frozen image to equal a second graph: all three failed, and we report them.

Slide 6 – Complementarity gain

On the slide. Table on the test split, mean of four replicas, in AUC points: fusion gain, control gain, CG and outcome for the three vision branches fused with W. Below: frozen vision, damage-trained heads, where the image helps.

What to say

How to read the table. Fusion gain: fused AUC minus the better branch. Control gain: the same for the stronger control, here a second training of W fused with W. CG: the difference (paper, Eq. 2).

The control. Fusing the graph with a second copy of itself already gains +1.63 points. That is the ensemble effect alone: no new information.

Frozen vision. Gains +1.25, *less* than the control: CG −0.38, negative in all four replicas. Against the GraphSAGE control it is equivalent (the difference is within the ±0.5-point margin we fixed). So with a strong graph the frozen image encoder adds nothing beyond an ensemble effect, while the raw +1.25 alone would have suggested it helps.

Damage-trained heads. They gain +2.79 and +3.66, exceeding the control by +1.16 and +2.03, positive in all four replicas: they carry information of their own. But even here the control explains about half of the gain, so the raw fusion gain overstates the image's contribution by roughly a factor of two.

Where the image helps. Splitting the queries: the CG is zero with 25 or 50% removed and becomes

positive only at 75%; it is largest on small plans (five rooms or fewer), where even the frozen branch passes the control. Reason: with few rooms left, the partial graph is compatible with many plans, while the image still keeps the shape of the hole. We also expected a larger CG for plans whose room graph is shared by many others; the data refuted it. This analysis is post hoc and on one seed: a description, not a confirmed result. It is consistent with slide 5: with graphs only as strong as the image, even the frozen branch passes the controls by 8 to 16 points.

If asked

Q: Why is the frozen CG negative?

With a graph near the ceiling, a second graph model corrects more of its remaining errors than the frozen image does.

Q: Do control gain and CG add up to the fusion gain?

Only as a summary: they come from two different fusions, so they describe the gain but do not split one fusion exactly.

Q: Are gains of one to four points too small to matter?

Small in absolute terms, but paired, with intervals excluding zero, and stable across four replicas. And their size is the point: it depends on graph strength, and half of it is ensemble effect.

Slide 7 – Conclusions

On the slide. Graph branch, complementarity, recommendation; limitations and future work.

What to say

Graph branch. Trained on partial plans and selected on robustness, the graph retrieves a plan from half of its rooms almost as well as from three quarters (95.4 vs 96.3 points at 50% and 25% removal). *Careful:* the slide (and the paper) say "as well as from the complete plan", but we never measure the complete plan (no 0% level); say "with a quarter missing".

Complementarity. The image adds information of its own only under two conditions: it is trained on the damage, or the graph is no stronger than the image.

Recommendation, beyond floor plans. Part of any fusion gain comes just from combining two models. Compare it with an ensemble control, here a fusion of two graph models, to measure what the second modality really adds.

Limitations. RPLAN only, with synthetic damages. Graphs come from exact records, not parsed from images, which also makes the relevance scores circular. Training budgets bind: the graph's best epochs are at the end of training, LayoutGKN was trained once, so both scores are lower bounds. One global fusion weight. The multi-damage head was trained once and tested only on the damages it saw. The "where it helps" analysis is post hoc, on one seed.

Future work. Graphs parsed from images, external datasets, a learned or per-query fusion, a damage the heads never saw, human relevance judgements.

If asked

Q: What would change with graphs parsed from images?

Not tested. Expected: a noisier, weaker graph, which by the inverted U leaves more room for the image; and no more circular relevance scores.

Q: Did you look at the test set?

Every number reported comes from pre-registered readings with all weights fixed on validation. Earlier stages of the project did read the test split with a weaker graph; none of those readings selects anything reported.